Herhalingsoefening: Smarthome

Beoordelingscriteria sourcecode

De student kan softwarecomponenten combineren tot een goed functionerend geheel.
De student kan kwaliteitsvolle applicaties ontwikkelen en implementeren.

Om aan deze vereisten in de studiegids te voldoen verwachten wij van de student:

- compileerbare en werkende code
- respecteer de Code Conventions van de programmeertaal
- code in commentaar wordt niet beschouwd als werkende code

De Smarthome toepassing

We ontwikkelen een smarthome toepassing waarmee we verschillende smart devices in huis kunnen aansturen. De gebruiker van de toepassing kan scenario's zoals 'Leaving home' en 'Coming home' maken waarmee meerdere smart home devices tegelijk bediend kunnen worden.

Deze afbeelding vind je in kleur terug in de resources folder van het project.

Je ontwikkelt een REST API in Spring Boot. Je start niet "from scratch", maar accepteert de github classroom assignment. Je implementeert enkele user story's. Bekijk ook de **README.md** voor extra info!

Gegeven

We maken voor onze implementatie gebruik van een H2 database. De tabellen worden automatisch gegenereerd wanneer de applicatie opstart.

Het domein-model vind je terug op de laatste pagina. De belangrijkste concepten in het domein-model zijn:

- **Room:** De kamers in het huis waar de smart devices aanwezig zijn.
- **Device**: Een sensor of apparaat kan aangestuurd of gecontroleerd worden door het smarthome systeem. Ieder smart device heeft een naam en een kamer. Devices hebben een type (DeviceType), momenteel zijn dat LIGHT, THERMOSTAT en DOOR_WINDOW_CONTACT. Enkel DOOR_WINDOW_CONTACT is een sensor. Verder hebben devices ook een status (DeviceState) zoals ON, OFF, OPEN, CLOSED of UNAVAILABLE. Het veld 'settings' wordt gebruikt om extra info bij te houden zoals de instellingen van de thermostaat. Tenslotte wordt het adres (UUID address) gebruikt door het smarthome systeem om met het device te communiceren.
- **Scenario**: een scenario heeft een naam en een lijst van acties voor het bedienen van smart home devices en het controleren van de status van sensoren.
- **Action**: een actie van een scenario. Wanneer een scenario wordt getriggered worden alle acties uitgevoerd. Een actie heeft een device, een gewenste status (deviceState) en eventueel settings. Bij een sensor wordt gecontroleerd of hij momenteel de gewenste status heeft. Wanneer het device geen sensor is, wordt (via de SmartHomeBridge) het device in de gewenste status gezet.

Gevraagd

☐ Vervolledig de entity klassen Room en Device. Enum-waarden worden steeds als een String opgeslagen. Gebruik voor alle primary keys de strategy IDENTITY. Voorzie het REST endpoint waarmee je alle devices kan ophalen: 'GET http://{server}:{port}/smarthome/devices' aanroepen (zie in de README.md). Maak gebruik van de DeviceDTO.

☐ Implementeer nu onderstaande user story's volgens de opgegeven richtlijnen.

Respecteer de 3-tier architectuur van de toepassing!

Titel: User story 1
Als gebruiker kan ik een scenario aanmaken.
Omschrijving: Als gebruiker geef ik de naam van het nieuwe scenario.

POST http://{server}:{port}/smarthome/scenarios (zie ook README.md)
JSON body:
{
`"name": "Leaving home"`
}

Responses: (zie ook README.md) - Bij succes: 201 CREATED - Bij fouten: 400 (BAD_REQUEST) of 409 (CONFLICT).
Acceptatiecriteria: ☐ De naam van het scenario is verplicht. Geef httpstatus 400 (Bad Request) als dit niet is ingevuld. ☐ Als er al een scenario bestaat met (exact dezelfde) opgegeven naam, dan wordt een DuplicateScenarioException opgegooid. Zorg dat deze exception httpstatus 409 (Conflict) geeft. Er wordt geen scenario opgeslagen in de databank. ☐ Als alle gegevens geldig zijn, wordt het scenario correct opgeslagen in de databank.
Implementatie-details: ☐ Annoteer de entity klasse Scenario. ☐ Je kan gebruikmaken van de record-klasse **ScenarioRequest** om de inkomende JSON te mappen naar een object. ☐ Maak een nieuwe RestController aan: **ScenarioController** waarin je het REST endpoint voorziet. ☐ Voorzie de repository **ScenarioRepository**. ☐ In de klasse **ScenarioService** implementeer je de business-regels. De exception klasse **DuplicateScenarioException** is voorzien.

Titel: User story 2
Als gebruiker kan ik een actie toevoegen aan een scenario.
Omschrijving: De gebruiker geeft het deviceId en de gewenste status van het device (deviceState) voor het scenario. De actie wordt opgeslagen bij het opgegeven scenario (scenarioId in de url).

PUT http://{server}:{port}/smarthome/scenarios/{scenarioId}/actions
(zie ook README.md)
JSON body:
{
"deviceId": 3,
"deviceState": "ON"
}

Responses: (zie ook README.md) - Bij succes: 202 (ACCEPTED) - Bij fouten: 404 (NOT_FOUND)
Acceptatiecriteria en implementatie-details: ☐ In de klasse ScenarioController voeg je het gevraagde REST-endpoint toe. Er hoeft geen validatie te gebeuren op de velden. ☐ Je kan gebruikmaken van de record-klasse **ActionRequest** om de inkomende JSON te mappen naar een object. ☐ Vervolledig de entity klasse **Action**. Zorg voor een **bi-directionele relatie** tussen de klassen Action en Scenario waarbij de klasse Action owner is van de relatie. Implementeer in de klasse Scenario de methode getActions() om de actions bij een scenario op te vragen. Zorg dat deze bi-directionele relatie correct wordt beheerd door de toepassing. (Tip: bekijk ook het klassendiagram!) ☐ In de klasse ScenarioService is de methode addAction al gedeeltelijk aangemaakt, vervolledig de implementatie volgens de richtlijnen in de code (zie TODO's in de code). ☐ Als er geen device bestaat met het opgegeven deviceId, dan wordt een NotFoundException opgegooid. Deze exception geeft httpstatus NOT_FOUND. De action wordt in dat geval niet opgeslagen. ☐ Als er geen scenario bestaat met het opgegeven scenarioId, dan wordt een NotFoundException opgegooid. Deze exception geeft httpstatus NOT_FOUND. De action wordt in dat geval niet opgeslagen. ☐ Als het scenario en het device bestaan, dan wordt een nieuw Action-object aangemaakt en opgeslagen in de databank.

Titel: User story 3
Als gebruiker kan ik een scenario triggeren.
Omschrijving: De gebruiker kan een scenario uitvoeren. Alle acties bij het scenario worden uitgevoerd. Als het device bij een actie een sensor is, dan wordt de status van de sensor gecontroleerd. Als de sensor niet de gewenste status heeft, dan worden de acties niet verder uitgevoerd en wordt de InvalidDeviceStateException opgegooid die resulteert in httpstatus 406 (NOT_ACCEPTABLE). Als alle sensoren de juiste status hebben, zullen alle andere acties van het scenario getriggered worden. Het smart device krijgt via de SmartHomeBridge een boodschap met de gewenste status en settings. De nieuwe status en settings worden opgeslaan in de databank. Als een device niet beschikbaar is, wordt de status van dat device UNAVAILABLE. Dit heeft dit verder geen gevolgen, de overige acties worden alsnog uitgevoerd.
POST http://{server}:{port}/smarthome/scenarios/{scenariold}/trigger (zie ook README.md)
Responses: (zie ook README.md) - Bij succes: 202 (ACCEPTED) - Bij fouten: 404 (NOT_FOUND) of 406 (NOT_ACCEPTABLE)
Acceptatiecriteria en implementatie-details: ☐ In de klasse ScenarioController voeg je het gevraagde REST-endpoint toe. ☐ In de klasse ScenarioService is de methode '*void triggers(Integer scenariold)*' al gedeeltelijk aangemaakt, vervolledig de implementatie volgens de richtlijnen in de code (zie TODO's). De methoden '*void checkSensor(Action action)*' en '*void trigger(Action action)*' zijn reeds volledig en correct geïmplementeerd. ☐ Als er geen scenario bestaat met het opgegeven scenariold, dan wordt een NotFoundException opgegooid. Deze exception geeft httpstatus NOT_FOUND. De action wordt in dat geval niet opgeslagen. ☐ Zorg dat de InvalidDeviceStateException de httpstatus 406 (NOT_ACCEPTABLE) geeft.

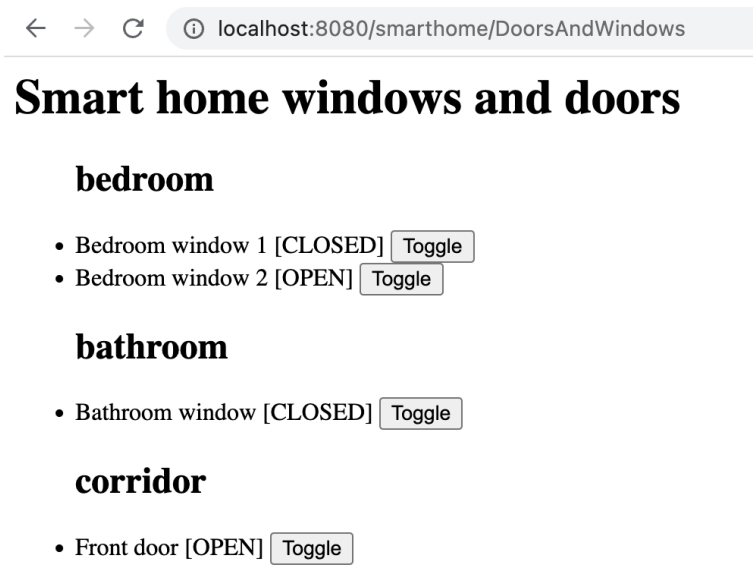
Titel: User story 4
Als gebruiker kan ik een overzicht krijgen van alle ramen en deuren en hun status.
Omschrijving: De pagina devices.html geeft per kamer een overzicht van alle raam-en deur-contacten (sensoren) met hun status. Via een knop kan de status van de ramen en deuren aangepast worden.
URL: http://{localhost}:{port}/smarthome/devices/byType?type={deviceType} **URL:** http://{localhost}:{port}/smarthome/devices/{deviceId}/toggle
Acceptatiecriteria: ☐ Een HTML-pagina met de status van alle sensoren voor deuren en ramen (deviceType is ROOM_WINDOW_CONTACT) wordt gegenereerd. Via de knoppen op de HTML-pagina kunnen we het sluiten en openen van ramen en deuren simuleren. De sensoren worden per kamer weergegeven.
Implementatie-details: De html-pagina devices.html is beschikbaar in folder resources/static.

☐ Voorzie in de repository **DeviceRepository** een methode (query) om alle Devices met het opgegeven DeviceType op te halen uit de databank. Roep deze methode aan in de methode ‘List<RoomDTO> getAllDevicesByDeviceType(DeviceType deviceType)’ in de DeviceService en geef het resultaat terug. Het resultaat bevat de verschillende kamers en per kamer de verschillende devices.

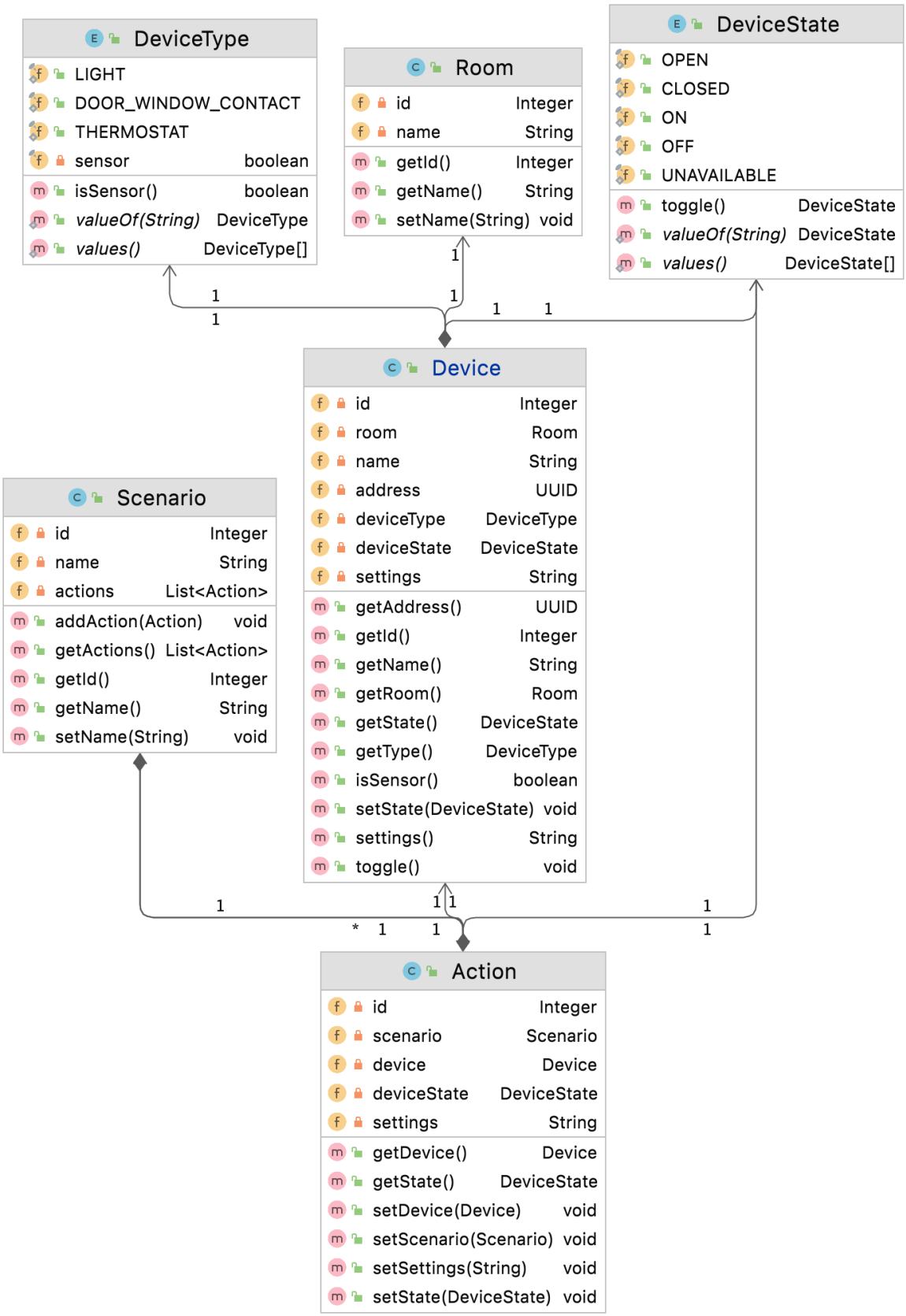
☐ Implementeer de tweede url om de status van een device aan te passen.

☐ Voorzie in de klasse Device de methode toggle() om de status van een device te veranderen. Schrijf 2 unit testen om deze methode te testen.

Voorbeeld html:



Veel succes!


DeviceType
LIGHT
DOOR_WINDOW_CONTACT
THERMOSTAT
sensor boolean
isSensor() boolean
valueOf(String) DeviceType
values() DeviceType[]
Room
id Integer
name String
getId() Integer
getName() String
setName(String) void
DeviceState
OPEN
CLOSED
ON
OFF
UNAVAILABLE
toggle() DeviceState
valueOf(String) DeviceState
values() DeviceState[]
Device
id Integer
room Room
name String
address UUID
deviceType DeviceType
deviceState DeviceState
settings String
getAddress() UUID
getId() Integer
getName() String
getRoom() Room
getState() DeviceState
getType() DeviceType
isSensor() boolean
setState(DeviceState) void
settings() String
toggle() void
Scenario
id Integer
name String
actions List<Action>
addAction(Action) void
getActions() List<Action>
getId() Integer
getName() String
setName(String) void
Action
id Integer
scenario Scenario
device Device
deviceState DeviceState
settings String
getDevice() Device
getState() DeviceState
setDevice(Device) void
setScenario(Scenario) void
setSettings(String) void
setState(DeviceState) void